第3回：交点計算のC#実装例

詳細解説 と 宿題の問題・解答 の固定例A〜Eを、7題材・155件の実行時自動チェックで確かめる.NET 10コンソール教材。

`double` のタプルで成分式・交差分類を手実装し、`System.Numerics.Vector2/Vector3` の `float` の結果と比較する。入力はソース内の固定値で、引数や標準入力は使わない。AutoCADライセンス・GUI・追加NuGetパッケージは不要。第1・2回のコードには依存しない。

必要環境

- **.NET 10 SDK**。対象フレームワークは `net10.0`。ランタイムだけではビルドできない。
- Visual Studioでは **Visual Studio 2026（18.0以上、Windows）** と「.NETデスクトップ開発」ワークロード、使用するVisual Studioに対応した.NET 10 SDKを使う。公式の導入・対応表 を参照。
- CLIでは.NET 10 SDKを導入したWindows・Linux・macOSのコンソールから実行できる。

Visual Studioで開いて実行する

1. 「プロジェクトやソリューションを開く」から IntersectionExamples.sln を開く。従来形式の `.sln` と `.csproj` が同じディレクトリにある。
2. ソリューションエクスプローラーのプロジェクトが `IntersectionExamples` 1件であることを確認する。必要なら右クリックで「スタートアッププロジェクトに設定」を選ぶ。
3. 構成を **Debug**、プラットフォームを **Any CPU** にする。
4. **Ctrl+Shift+B** でソリューションをビルドする。
5. **Ctrl+F5** でデバッグなし実行、**F5** でデバッグ実行する。全件成功時、最後に「すべての 155 チェックに成功しました。」と表示される。

F9／F11で交点の条件を追う

1. Program.cs の `Lines2D` にある `double denominator = Cross2(r, s)` の行に **F9** でブレークポイントを置く。
2. **F5** で停止し、「ローカル」「ウォッチ」で `r=(4,4)`、`s=(4,-4)`、`w=(0,4)` を確認する。
3. **F11** で IntersectionMath.cs の `Cross2` に入り、`4*(-4)-4*4=-32` を追う。**Shift+F11** で戻り、**F10** で分子－16・－16を確認する。
4. `LineLine2` の呼び出しから **F11** で入り、`denominator`、`numeratorT`、`numeratorU`、`t=.5`、`u=.5`、両方から求めた位置 `(2,2)` を追う。メソッド名にカーソルを合わせるとXMLコメントの入力・結果・前提を読める。

`Segments2D` では `SegmentSegment2` に入り、延長線の交点でt=1.5が範囲外になる分岐を追う。同一直線の例では `t0=.5`、`t1=1.5`、`low=.5`、`high=1` と、共有部分の両端 `(2,0)`・`(4,0)` を確認する。零長線分は `Degenerate` で止まり、平行と誤分類しない。

`LineAndPlane` では `LinePlane3` の呼び出しへブレークポイントを移す。F11で `n・d=24`、`n・(A-P)=12`、`t=.5` を追う。方向を `(1,0,0)` にした例では分母0で割らず、分子12なら `None`、0なら `Coincident` を返すことを確認する。

`TwoPlanes` の `PlanePlane3` では、法線の単位化・外積 `v`・自由に選ぶ座標 `free`・2元連立式の `determinant`・通過点 `x` を追う。固定例EではYを0と選び、X=1・Z=2を解く。方向だけでは位置を決められないことに注目する。

コンソール出力を読むにはCtrl+F5を使う。F5で終了前に止める場合は `Main` の成功メッセージへブレークポイントを置く。CLIでは起動元の端末に出力が残るため、待機用の `ReadKey` は不要。

CLIで実行する

リポジトリのルートから実行する。

```
dotnet build sessions/03-intersection/examples/IntersectionExamples.sln -c Debug
dotnet run --project sessions/03-intersection/examples/IntersectionExamples.csproj -c Debug
--no-build
dotnet build sessions/03-intersection/examples/IntersectionExamples.sln -c Release
dotnet run --project sessions/03-intersection/examples/IntersectionExamples.csproj -c
Release --no-build
```

この `examples/` を作業ディレクトリにした場合は `dotnet run` でもビルド・実行できる。全件成功は終了コード0。不一致・非有限な比較値・期待しない分類は `FAIL` と検証ラベルを出して終了コード1になる。数値の失敗では成分番号・実値・期待値・許容誤差も表示する。

ファイルと読む順序

ファイル	内容
IntersectionExamples.sln	Visual Studioで開く入口。第3回専用プロジェクト1件
IntersectionExamples.csproj	net10.0のコンソール設定。追加パッケージなし
Program.cs	固定入力、手計算の期待値、API比較、残差・分類・範囲のチェック
IntersectionMath.cs	double成分式、直線・線分・平面の条件と分岐、連立式
IntersectionResults.cs	2D／3Dタプルの型別名、結果の分類、null可能な結果フィールド

`Program.Main` → 各題材の入力とコメント → `IntersectionMath` の式・分岐 → 元の題材の期待値と残差の順に読む。同じ直交座標系を使い、2DはX右・Y上、3Dは右手系。方向は終点までの変位で、媒介変数は無次元とする。

実行順・入力・期待結果

題材・メソッド	解説／宿題との対応	主な入力と期待値	チェック数
1. ParameterDomains	②／基本1-1,4	P=(0,0), r=(4,4)。t=0,1,.5,1.5,－.5の位置、閉区間と範囲外	8
2. Lines2D	③／基本1-2,3	固定例A：分母－32、分子－16/－16、t=u=.5、交点(2,2)。原点が正しい交点の追加例	9
3. Segments2D	④／基本2	Aの内部、Cの範囲外t=1.5と端点t=1。Bの平行／一致、同一直線の非重複／端点／共有区間(2,0)〜(4,0)、逆向き・零長	34
4. TrianglePlane	⑤／基本3-1	A=(0,0,2), B=(2,0,2), C=(0,3,2) → n=(0,0,6)、平面z=2。一直線・重複点・小面積を拒否	10
5. LineAndPlane	⑥／基本3-2〜5	固定例D：P=(1,2,0), d=(2,0,4)、分子12・分母24、t=.5、交点(2,2,2)。平行で別／平面内、線分外t=4、端点t=0/1	18
6. TwoPlanes	⑦／発展	固定例E：z=2とx=1、方向(0,1,0)、通過点(1,0,2)、交線(1,t,2)。平行別面／一致、自由座標の3分岐と斜めの法線	20
7. PrecisionAndExceptions	⑧／退化の補足	ゼロ・非有限値・ほぼ平行・近い平行図形・floatの丸め・倍率／単位・0/1境界・桁あふれ／アンダーフロー	56

題材6の追加例は、x=1とy=2で通過点(1,2,0)、y=2とz=3で(0,2,3)、x+2y=1とy+3z=－1で(0,.5,－.5)。固定例Eだけに専用化せず、各通過点とその方向へ進んだ点を両平面式へ戻す。

出力例（抜粋）：

```
2. 2D無限直線の交点
  [OK] A: 分母・分子T・分子U: -32, -16, -16
  [OK] A: t/u/交点: 0.5, 0.5, 2, 2
...
  [OK] C: 重複区間: actual=OverlapSegment, expected=OverlapSegment
  [OK] C: 重複区間の両端: 2, 0, 4, 0
...
  [OK] D: tと交点: 0.5, 2, 2, 2
...
  [OK] E: 通過点: 1, 0, 2
  [OK] E: 交線方向: 0, 1, 0
...
すべての 155 チェックに成功しました。
```

結果の種類と値の使い方

Kind	意味と利用できるフィールド
Point	唯一の共有点。PointとT、2DではUも持つ
None	有効な図形に共有点なし。座標・変数はnull
Coincident	無限直線どうしなら直線一致、直線×平面なら直線全体が平面内、平面×平面なら平面全体が共通。座標はnull
OverlapSegment	2D閉線分の区間重複。OverlapStartとOverlapEndの両端を持つ。Point・T・Uはnull
Line	2平面の交線。Pointは交線上の代表点、Directionは方向。Tはnull
Degenerate	ゼロ方向・零長線分・ゼロ法線、または教材の方向長の下限以下。座標はnull
InvalidInput	座標・方向・法線にNaN／Infinityを含む。座標はnull
Uncertain	ほぼ平行、判定境界付近、非有限な中間値、残差超過などで保留。Noneとは異なり、座標はnull

Kindを確認してから対応するフィールドを使う。`Point=null` を `(0,0)` や `(0,0,0)` で置き換えない。原点が正しい唯一の交点になる場合もあり、座標だけでは「なし」と区別できない。

零長線分はこのサンプルでは明示的にDegenerateとする。 講義・宿題で説明した点-線分処理は実装しない。両方が点なら同一点か、片方が点なら他方の線分上かを別に調べる必要があり、平行／一致に混ぜない。

`TryTriangleNormal` は平面を作れるかをboolで返す。falseには退化・下限以下・非有限値などを含み、失敗時のout値を平面の法線として使わない。直線×平面のAPIは無限直線用で、有限線分として扱う場合はPointのTが閉区間内かを呼び出し側で確認する。平面内の線分なら共有部分は線分全体となる。

数式とAPIの対応

- `Cross2` は自作のスカラー成分式。`LineLine2` はw=tr－usからt=cross(w,s)/cross(r,s)、u=cross(w,r)/cross(r,s)を計算し、両方の式から求めた位置の差を検査する。
- `SegmentSegment2` は一点ならt・u両方の範囲を確認。同一直線なら第2線分の両端を第1線分のtへ射影し、区間の共通範囲を求める。重複両端の順序は第1線分のt順で、X座標の昇順とは限らない。
- `LinePlane3` はn・(P+td－A)=0から分子n・(A－P)、分母n・dを求める。分母0なら位置の条件へ分岐し、一点を返す前に平面残差を検査する。
- `PlanePlane3` は両法線を単位化してから外積vを求める。vの最大絶対成分に対応する座標を0と選び、残る2座標を2元連立式で解く。得た通過点を両平面へ代入して検査する。方向は元の法線外積の非ゼロ実数倍で、固定例Eではそのまま(0,1,0)。有限面の境界処理は含まない。
- Vector3.Cross と Vector3.Dot を成分式と比較する。2DはZ=0へ埋め込んだ外積のZ成分と照合する。Vector2/3の各成分はfloatであり、doubleの手計算とは精度を分ける。

しきい値・退化・数値精度

条件	教材の値と尺度
数値の期待値比較	doubleは絶対誤差1e-12、floatを含む比較は1e-5
方向長の下限	MinimumDirectionLength=1e-12、座標と同じ長さの単位
三角形の2倍面積の下限	MinimumTriangleArea2=1e-12、長さの2乗。辺長と角度も検査
角度の尺度	AngularTolerance=1e-12、無次元。2Dはabs(cross)/(長さの積)、直線×平面はabs(n・d)/(長さの積)、2平面は単位法線の外積の長さ
距離・残差	DistanceTolerance=1e-9、座標と同じ長さの単位。法線や方向の長さで補正
線分の範囲外の保留帯	ParameterTolerance=1e-12、無次元。範囲外でも境界との差がこの値以下なら保留

分母の絶対値に共通epsilonを直接当てない。非ゼロ分母でも角度の尺度が小さいときはUncertainとし、厳密な平行へ読み替えない。計算上の分母が0なら、さらに位置のずれを調べる。ずれの距離が厳密に0なら一致、非ゼロでも距離しきい値以下なら保留とする。

線分の範囲は端点を含む。ごく小さい範囲外を端点へ丸めずUncertain、十分に範囲外ならNone、厳密な0/1や内側ならPointとする。位置の同じずれでもtの差は方向長に依存するため、パラメータの許容値を距離の許容値と混同しない。

これらは教材の固定単位・小さな入力向けの方針で、数学的な厳密判定ではない。計算上の0にも丸め誤差があり、全ての不安定な入力を検出できるわけではない。極端な値には長さの2乗・内積・外積で桁あふれやアンダーフローがあり、扱えない中間値は保留する。大きな共通座標を引く桁落ちや、境界の厳密な位相判定には頑健な述語・誤差評価・スケーリングなどが別途必要になる。

題材7では方向を1e-6／1e6倍し、交点は同じでもt・uの尺度が変わることを示す。法線の倍率と符号では平面の点集合は変わらない。同じ固定例をmmからmへ換算してもt・uは変わらないが、実務で単位を変えるときは長さ・距離・面積のしきい値も合わせて見直す。法線は任意にスケールできるため、方向長の下限をそのまま適用せず、ゼロ・有限性・長さ計算の可否を検査する。

`1+1e-13` の差はdoubleには残るがfloatへ丸めると消え、外積値が変わる。floatからdoubleへ戻しても失われた差は復元できない。比較は表示前の値で行い、表示は通常G9、微小な分母はG17とする。

自動チェックとCI

実行時の155チェックは、数値・残差・分類と値の有無・範囲の各検証呼び出しを1件として数える。題材内の入力反復もそれぞれ数える。7題材の件数8／9／34／10／18／20／56、実際の `[OK]` 155件、最後の成功要約、終了コード0が揃うことを確認する。`Check` は実値と期待値の両方の有限性を確認し、NaN比較やInfinityの差による見逃しを避ける。

Check Intersection Examples は、第3回のC#コード（`.cs`）・プロジェクト（`.csproj`）・ソリューション（`.sln`）・ビルド設定（`.props`／`.targets`）、ルート共通設定の `global.json`／`Directory.Build.*`、ワークフロー自身の変更を、PR時とmainへのpush時に検証する。Markdownだけの変更では起動しない。

Ubuntu 24.04・.NET SDK 10.0.401でソリューションをrestoreし、Releaseビルドして実行する。終了コード0、実際の `[OK]` 155件、期待する成功要約の3点を確認する。途中で失敗した場合も実行コマンドの失敗を伝播する。チェック数を変える場合は題材の内容をレビューし、ワークフローの `EXPECTED_CHECKS` と、このREADMEの件数・出力例を同じPRで更新する。

PRのChecks、または **Actions → Check Intersection Examples → 対象の実行 → IntersectionExamples (Release)** で確認できる。失敗時は `Restore solution`・`Build solution (Release)`・`Run IntersectionExamples and verify all 155 checks` の該当ログを開く。`FAIL` のラベルから式・分類・実値・期待値・許容誤差を確認する。件数や成功要約の不一致は `::error::` で表示する。

CIの権限は `contents: read` のみで、PAT・Secret・書き込み権限は不要。チェックアウトした認証情報を保持せず、ソースやPDFの自動コミットも行わない。第1回43件・第2回73件の専用CI、PDF生成とは独立している。

資料の元データ（GitHubリポジトリ）： https://github.com/mkunori/geometry-study-group